Response to Reviewer

We thank the reviewer for an exceptionally thorough and constructive report. The review has materially improved the paper, above all by prompting us to re-analyze the data under the paired/blocked structure of our own design. Below we respond to every point. Section and table references are to the revised manuscript; the revised submission build is `ist/ist_main.tex` and a numbered online supplement accompanies it.

A clarification that resolves several points at once

The review was conducted on `express_db_access-3.pdf`, which is our **generic 'article' build**, used internally for quick iteration. The **submission build** (`ist/ist_main.tex`, the Elsevier `elsarticle` version) already carried the items flagged as missing: a structured abstract under Context / Objective / Methods / Results / Conclusion, a keyword list, and CRediT, competing-interest, funding, and generative-AI declarations. We should have supplied only the submission build, and apologize for the confusion. To ensure no circulated version is ever non-compliant, we have now brought the generic build to full front-matter parity as well. This resolves §5.1, §7.2, and the manuscript side of §9 and §15.

Major concerns

§6.1 — Tests did not match the paired design. This is the most important correction and we are grateful for it. The reviewer is exactly right: within each replicate every layer is driven by the identical request stream, so layer comparisons are paired and the unpaired Mann–Whitney/Cliff's analysis discarded that structure. We re-analyzed the same 25-replicate data under the paired design (no re-measurement was needed, since the sample arrays are index-aligned by replicate). Adjacent layers are now compared on their per-replicate throughput ratios with a two-sided **paired sign-flip permutation test** (20,000 permutations) cross-checked with the **Wilcoxon signed-rank test**; we report the geometric-mean paired ratio with a paired bootstrap CI and paired dominance (Table 11, Section 4.6, Methodology §3.6). As the reviewer anticipated, the large gaps survive overwhelmingly (six of seven adjacent deep-fetch pairs separate at the permutation floor, $p \leq 5\times10^{-5}$, with the faster layer ahead in every replicate), **and the close pg–Prisma conclusion changed**: the paired test no longer resolves them as different (ratio 1.02, $p = 0.058$; Wilcoxon $p = 0.08$), while a paired TOST confirms equivalence within ±5% (90% CI [1.004, 1.037]). We now describe the top of the PostgreSQL deep-fetch ranking as a practical tie rather than a significant difference. The layer×engine interaction test (§4.4) was likewise rebuilt as a **blocked permutation test** that permutes layer labels within replicate blocks (fixing both the pooled-residual permutation and the sub-sampling the reviewer flagged in §8/§7.12).

§6.2 — "Independent replicates" overstated. Corrected throughout. Methodology now states the experimental unit explicitly (the freshly booted server run for one cell in one replicate; HTTP requests are subsamples), says that a fresh process "prevents persistent application-process state from carrying across replicate runs" rather than guaranteeing independence, and treats replicate index and measurement order as blocking factors. The Threats "Conclusion validity" paragraph was revised in the same spirit.

§6.3 — Causal attribution of the same-SQL contrast too strong. Agreed and corrected in all loci (abstract, Results §4.2, Discussion, Conclusion, and the Table 9 caption). We now describe the same-SQL control as a **composite-path contrast**: switching from a layer's idiomatic relational-fetch path to the common raw-SQL-and-shared-mapping path changes query strategy, query count, prepared-statement behavior, the raw-versus-ORM API, and result mar-

shalling together, so it does not isolate hydration. What it establishes is that the raw-execution component is small and the idiomatic overhead lies upstream of it.

§6.4 — Application-side CPU too close to total efficiency. Corrected. We now report application, database, and combined app+database CPU per completed request (Supplement Table S5), state that CPU is normalized to one logical core, and use load-generator CPU only as an observer-capacity check. The reframing materially changes the efficiency story and we report it honestly: pg leads application-tier efficiency ~5× over Prisma (3,741 vs 740 req/app-CPU-s) but only ~1.4× end-to-end (813 vs 596 req/combined-CPU-s), because pg pushes ~3.6 database cores that the application-only figure ignored. The "≈498% application CPU" fact is unchanged; the "CPU subsidy" language is now tier-specific.

§6.5 — Single-host resource interference. We added a resource-isolation check (Threats, External validity): the representative deep-fetch cells were re-run with the application, database, and load generator pinned to disjoint core sets. The isolated and shared-host medians agree within ~2% (pg 0.98, Prisma 1.01, MikroORM 1.00) with the ranking unchanged, so same-host contention is not driving the result. The automated environment capture now records CPU governor, NUMA topology, virtualization (the host is a 16-vCPU single-NUMA VM, disclosed), and process affinity. We were unable to add a second physical machine, and say so.

§6.6 — "Neutral" needs qualification. Replaced "neutral" with "vendor-independent" throughout (and the prior-art table column "Neutral?" with "Independent?"). Methodology now states that vendor independence means authorship (no evaluated library's maintainers were involved, and none was invited to inspect the adapters) and explicitly does not remove the consequential design choices any benchmark makes. The competing-interest declaration already affirms no relationship with the evaluated projects.

§6.7 — Workload representativeness. We now call these "five selected access patterns ... not a sample from a measured production workload distribution" (Introduction, Methodology) and added a transactional multi-statement write experiment (`POST /threads`: a post and five comments in one transaction) for a representative layer of each tier on PostgreSQL (Supplement Table S8). Its between-layer spread is 3.3×, close to the single-row insert and far below the 6.6× read spread, and the ordering tracks the single-row insert, so the write conclusions extend to a transactional pattern. Update/delete and bulk patterns remain future work, now stated as such.

§6.8 — Pool-size fairness. We reframed the fixed pool of 10 as an equal-configuration comparison (not per-layer-tuned) and added a pool-size sensitivity sweep from 1 to 50 for a representative layer of each tier (Supplement Table S7): the three-band ordering is preserved at every pool size and each layer saturates well below 50, so the fixed-pool choice does not drive the ranking.

§6.9 — Tail-latency estimand, counts, uncertainty. Methodology now states that the reported p99 is the median of per-run p99s; we report bootstrap CIs on the per-replicate p99 (Section 4.6), the per-run request count, and the error/timeout/non-2xx counts per cell (all zero in the primary matrix's measurement windows). The open-loop harness uses HdrHistogram (sub-millisecond resolution); p99 values are rounded to the millisecond in the tables.

§6.10 — Over-reliance on the supplement. The supplement is now a numbered companion (Supplement Tables S1–S8), submitted with the manuscript and archived under the same Zenodo DOI; every previously vague "online supplement" mention in the main text is now a specific "Supplement Table S#" reference.

Minor concerns (§7)

- 7.1 Title: we retained "for PostgreSQL and MySQL"; we are happy to change to

"Across" if the editor prefers.

- 7.3 "Full taxonomy" removed (it survived only in the generic abstract); we say "a broad cross-section of the three tiers".
- 7.4 We now classify each library by its dominant interface and state the criteria; Drizzle is placed at the query-builder end of the ORM range and Objection as an ORM over Knex.
- 7.5 We now say the versions were "selected and installed on the measurement date" and cite the lockfile and automated environment capture.
- 7.6 Express 4: we acknowledge that a framework version could interact with a layer's scheduling or serialization, not merely shift a common floor.
- 7.7 Renamed to "fixed-payload endpoint baseline" and noted it bounds framework and serialization cost, not per-request object construction.
- 7.8 / 7.10 We report robust relative dispersion (relative MAD) alongside CV and removed the awkward "orders of magnitude above CV" comparison.
- 7.9 "Conventionally powered" deleted; the conclusions rest on effect sizes and interval estimates, not an asserted power.
- 7.11 Rank correlations are now presented descriptively over the seven evaluated systems (not a population sample), with no population confidence intervals.
- 7.12 We report the permutation count (20,000) and that $p = (ge+1)/(B+1)$, with the floor written as $p \leq 5\times10^{-5}$.
- 7.13 We label the seven adjacent-rank comparisons as the confirmatory family (Bonferroni-corrected) and treat other comparisons as descriptive.
- 7.14 The $\pm5\%$ margin is now described as an a-priori chosen practical-equivalence threshold (below the study's own run-to-run CV), not a preregistered one.
- 7.15 The Figure 1 caption and the two echoing sentences now say the three-band ordering holds at $\geq$32 connections, matching the Results body.
- 7.16 Practical guidance is bounded: "safer default for throughput and tail latency", with an explicit note that productivity, type safety, correctness, maintainability, and security are not measured and may outweigh throughput.
- 7.17 "Catastrophic" replaced by a quantitative collapse criterion (achieves under half the offered rate with a majority of requests timing out).
- 7.18 All 70 references were machine-verified against Crossref/DOI resolution; no substantive discrepancy was found. Two cosmetic items were polished.

Novelty and search (§10)

We softened the universal novelty claims to "we did not identify a study combining all four properties in the sources searched through July 2026" (Positioning) and archived the search protocol (databases, query terms, window, screening) in the replication package.

Presentation (§11)

We reduced the manuscript length by roughly 20%, de-duplicated the recurring figures (the deep-fetch spreads, Prisma's CPU, the MySQL write spread, the same-SQL collapse, the novelty axes) to a single authoritative statement each with back-references, split the longest multi-claim sentences, and shortened the conclusion so that it synthesizes rather than re-lists results.

Answers to the reviewer's questions (§13)

1. The ±5% margin and the adjacent-rank comparisons were fixed a priori (before

inspecting results) but were not preregistered; we describe them as chosen, not preregistered, and the repository history is available.

1. The unpaired test was an oversight; it is corrected to a paired analysis.
2. The Freedman–Lane permutation unit is now the replicate block (layer labels

permuted within each replicate).

1. 20,000 permutations; p = (ge+1)/(B+1); the minimum attainable value is $\approx 5 \times 10^{-5}$.
2. The primary campaign did not pin cores; the isolation re-run (§6.5) does, and we

disclose both.

1. CPU is the full process tree (parent + descendants) from /proc; Prisma's engine

is an in-process Node-API library (no child processes, ~30 threads).

1. Combined app+database CPU-seconds per request are now reported (Supplement S5).
2. No HTTP errors, connection failures, or timeouts occurred in the primary 90-cell

matrix's measurement windows; counts are reported per cell.

1. Each run-level p99 is over $\approx$ rps $\times$ 12 s requests; we report the count.
2. autocannon uses HdrHistogram (sub-ms); tables round p99 to the millisecond.
3. The same-SQL control uses identical parameter values and the same two

statements (byte-identical, captured server-side); we disclose the differing wire protocols.

1. "Idiomatic" is each library's documented recommended mechanism; maintainers

were not invited to inspect the adapters, now stated as a limitation.

1. The tuned native variants are labelled lower-bound baselines; ORM tuning is

open-ended, and the pool-size sweep addresses the fairness concern directly.

1. Prisma 5.22 embeds the Rust query engine our CPU accounting characterizes;

version 7's TypeScript rewrite is disclosed as a validity caveat.

1. The complete numbered supplement is archived with the same DOI as the code and

data.

1. Code is MIT-licensed; text and data are CC-BY.
2. Generative-AI use is declared in the manuscript.

We believe the revision addresses every essential and strongly-recommended point, and we thank the reviewer again for the depth of the report.